1 Method

We introduce **GradMill**, a differentiable-simulation approach to CNC toolpath generation. Rather than learning a control policy, GradMill optimizes a discrete tool trajectory directly: a sequence of per-step cutter displacements is treated as the optimization variable, and gradients of a terminal geometry loss are backpropagated through a differentiable constructive-solid-geometry (CSG) carving simulator into the trajectory. The system is implemented in Taichi with `ti.ad.Tape` autodiff.

A central finding of this work — and the design principle that follows from it — is that the simulator's gradients, while correctly computed, optimize a *differentiable surrogate* whose optimum is misaligned with the *deployable* (boolean) machining outcome. Directly following the gradients degrades the deployable metric. Our method therefore couples gradient-based optimization with (i) a geometry-informed trajectory initialization that encodes domain knowledge, and (ii) a best-checkpoint selection protocol that preserves the deployable optimum against surrogate-driven degradation. We describe each component below.

1.1 Problem Formulation

The machine work volume contains a rectilinear stock block (a normalized $[0,1]^3$ cube, default 1 in^3 at 0.5 mm voxels) and a target part S_{tgt} defined as the negative interior of an analytic signed distance field (SDF) $\phi_{\text{tgt}}(\mathbf{p})$ (sphere, cylinder, box, or pyramid). A cylindrical end mill of radius r and flute height h, surmounted coaxially by a wider holder (the collet/spindle), removes material by sweeping through the stock. A trajectory of T tool positions $\{\mathbf{x}_0, \ldots, \mathbf{x}_{T-1}\}$ produces a carved stock $S_{\text{carved}} \subseteq S_{\text{stock}}$. The objective is a trajectory whose carved geometry matches the target, scored by the Dice coefficient between the remaining material S_{carved} and the part S_{tgt}.

1.2 Differentiable CSG Carving Simulator

Stock and target representation. The stock is a dense 3D scalar SDF field $\Phi_t(\mathbf{p})$ on a regular voxel grid (32^3 default), where $\Phi < 0$ denotes interior (remaining material). At $t = 0$ the stock is the full envelope: $\Phi_0 = \phi_{\text{box}}$. The target is an analytic SDF ϕ_{tgt} evaluated pointwise (sphere, cylinder, square prism, or square pyramid), so no target discretization is needed for the loss. Distances are measured in voxel units, yielding an isotropic, physically-faithful metric.

Tool signed distance. The cutter is a swept cylinder of radius r and height h. For the t-th cut, spanning segment $\mathbf{x}_t \to \mathbf{x}_{t+1}$, the tool SDF at a query point $\mathbf{p}$ combines the in-plane distance to the swept capsule axis with the axial extent:

$$\phi_{\text{tool}}(\mathbf{p}, t) = \text{smooth\_union}(d_{xy}(\mathbf{p}, t),\ d_z(\mathbf{p}, t);\ k)\,, \tag{1}$$

where d_{xy} is the point-to-segment distance in xy minus r, d_z measures the axial band centered at $z_{\text{base}}(t) + h/2$ with half-extent $h/2$, and the tool's base z is interpolated along the segment. The smooth union uses the log-sum-exp smooth maximum

$$\text{smooth\_max}(a, b; k) = \tfrac{1}{k} \log\left(e^{ka} + e^{kb}\right), \tag{2}$$

with sharpness k (default 10). A separate holder SDF, mirroring this geometry with the holder radius and a z-range beginning at the tool top, models the spindle above the flutes.

Differentiable carving. Each cut updates the stock by a smooth boolean union, removing the tool's interior from the remaining material:

$$\Phi_{t+1}(\mathbf{p}) = \text{smooth\_max}(\Phi_t(\mathbf{p}),\ -\phi_{\text{tool}}(\mathbf{p}, t);\ k)\,. \tag{3}$$

The whole unrolled loop — position reconstruction, carving, and loss — runs inside a single `ti.ad.Tape`, so $\partial\mathcal{L}/\partial\mathbf{x}_t$ flows back through every cut. Because each step's smooth maximum adds a bias of $\mathcal{O}(\log 2/k)$ per union, the differentiable carve *over-erodes* relative to the exact boolean; we return to this in Sec. 1.6.

Speed-limit constraint. Real machines clamp feed and traverse rates. We model this with a differentiable per-step clip: the commanded displacement $\boldsymbol{\delta}_t$ implies a speed $|\boldsymbol{\delta}_t \cdot \mathbf{L}|/\Delta t$ (mm/s, with $\mathbf{L}$ the per-axis envelope). If it exceeds the regime cap, $\boldsymbol{\delta}_t$ is scaled down to run exactly at the cap (direction preserved). The regime is *feed* (cutting speed, when the destination probes within a safe distance of remaining stock) or *rapid* (traverse, when clear). This is a hard gate with zero gradient through the regime selection itself, while gradients still flow through the clipped magnitude into $\boldsymbol{\delta}_t$. Position reconstruction is interleaved with carving (each step is clipped against the stock as it evolves at the start of that step), so the constraint couples to the geometry.

1.3 Trajectory Parametrization

The trajectory is parametrized by $T-1$ per-step displacements $\{\boldsymbol{\delta}_t\}_{t=0}^{T-2}$, with the first tool position fixed at a canonical start $\mathbf{x}_0 = (0.5, 0.5, 1.0)$ (above the stock center). Positions are the cumulative clipped sum $\mathbf{x}_{t+1} = \mathbf{x}_t + \text{clip}(\boldsymbol{\delta}_t)$. Parametrizing displacements (rather than absolute positions) lets the optimizer shape motion directly and keeps the speed-limit constraint a simple function of the variables. The parameters are stored as a PyTorch tensor with `requires_grad=True` and optimized by Adam.

1.4 Loss Function

The terminal geometry loss is computed on the final stock Φ_T against the target. SDFs are converted to occupancy via a sigmoid $\sigma(\cdot)$ with a 1-voxel scale, giving

smooth, differentiable occupancy fields $o_{\text{stock}} = \sigma(\Phi_T)$ and $o_{\text{tgt}} = \sigma(\phi_{\text{tgt}})$. Two error terms are defined at each voxel:

$$\text{gouge} = o_{\text{tgt}}\,(1 - o_{\text{stock}}) \quad \text{(part material removed)}, \tag{4}$$

$$\text{residual} = (1 - o_{\text{tgt}})\,o_{\text{stock}} \quad \text{(waste material left)}, \tag{5}$$

and the objective is their weighted sum of squares,

$$\mathcal{L}_{\text{geom}} = \frac{1}{N} \sum_{\mathbf{p}} \left[\, w_{\text{gouge}}\,\text{gouge}^2 + w_{\text{residual}}\,\text{residual}^2 \,\right]. \tag{6}$$

The residual term rewards removing waste; the gouge term penalizes cutting into the part. Optional differentiable barriers (all inactive in the final operating point) are added to $\mathcal{L}$: a one-sided holder-penetration barrier relu(margin $-$ $\phi_{\text{holder}})^2$ that is zero with zero gradient whenever the holder has clearance, and several trajectory regularizers (path-length, jerk, air-cut, contour-hug). These were explored and ultimately disabled (Sec. 1.6), as they trade off the geometry objective without improving the deployable metric.

1.5 Optimization

Each iteration pushes the current displacements into the simulator, runs the differentiable forward pass inside `ti.ad.Tape`, reads the resulting $\partial\mathcal{L}/\partial\boldsymbol{\delta}$, and takes an Adam step. The per-iteration gradient ℓ_2-norm is clipped to a threshold (default 0.5) to stabilize optimization. The learning rate is the most sensitive hyperparameter (Adam, $1{\times}10^{-3}$): higher rates overshoot the carving basin and degrade dice, while lower rates underfit within the compute budget.

The simulator gradients are thus computed and followed at every iteration. The remainder of this section explains why following them, alone, is insufficient, and how the method is structured around that fact.

1.6 The Soft/Hard Carve Gap

The differentiable carve (Eq. 3) uses `smooth_max`, so its per-step over-erosion accumulates over T cuts. The deployable machining outcome is instead an *exact* boolean union,

$$\Phi_{t+1}^{\text{hard}}(\mathbf{p}) = \max\Big(\Phi_t^{\text{hard}}(\mathbf{p}),\ -\phi_{\text{tool}}^{\text{sharp}}(\mathbf{p}, t)\Big), \tag{7}$$

using a sharp (non-smoothed) tool SDF. The training loss $\mathcal{L}_{\text{geom}}$ is evaluated on Φ_T (soft), but the *deployable* metric — the Dice coefficient reported and used for checkpoint selection — is evaluated on Φ_T^{hard}.

This creates a structural decoupling: the soft loss rewards removing more material, and because the soft union over-erodes, a trajectory that minimizes $\mathcal{L}_{\text{geom}}$ systematically over-removes material relative to the target. Under the exact boolean carve, that over-removal *gouges* the part. Consequently, following the simulator gradients lowers the soft loss while *lowering* the deployable

Dice. We verify this empirically: from a strong initialization, soft optimization collapses hard Dice on every shape tested (e.g. sphere $0.93 \to 0.56$, cylinder $0.94 \to 0.72$, pyramid $0.83 \to 0.41$ over training). Sharpening the surrogate ($k \to 5$) does not help — gradients vanish as the log-sum-exp saturates. Loss-based air reduction and contour-hugging regularizers fare no better: they trade off the geometry objective for marginal, non-transferring soft gains.

The implication is that the differentiable simulator's value is not in the *direction* of its gradients for final refinement, but in (a) providing a smooth, jointly-defined objective over a structured trajectory space, and (b) enabling a search procedure around an informed starting point. Our method therefore invests in initialization and selection rather than in loss shaping.

1.7 Geometry-Informed Initialization

Because the deployable optimum is determined by the initialization rather than by gradient refinement, the init encodes the bulk of the method's domain knowledge. We use a *z-level finishing* initialization (`init_mode=zlayer`) that exploits the tall-tool geometry: the cutter spans $[z_{\text{base}},\, z_{\text{base}}{+}h]$ upward from its commanded base, so descending the base from above the stock to below it sweeps each z-level's waste *annulus* without the high base ever reaching the part equator.

At each z the tool orbits at a radius that oscillates between a *surface-offset safe radius* $r_{\text{safe}}(z) = r_{\text{surf}}(z) + r + \text{margin}$ (just outside the target surface, so the cutter's inner edge clears the part) and the cube wall r_{outer}, sweeping the waste annulus densely:

$$r_{\text{orbit}}(t) = r_{\text{safe}} + (r_{\text{outer}} - r_{\text{safe}})\left(\tfrac{1}{2} + \tfrac{1}{2}\sin(2\pi\,\omega\, f_t)\right), \tag{8}$$

$$\boldsymbol{\theta}(t) = 2\pi\, R\, f_t, \quad f_t = t/(T{-}2), \tag{9}$$

with angular revolutions R and radial oscillation cycles ω as shape-tuned parameters. The safe radius and orbit shape are shape-aware:

- **Sphere / cylinder:** $r_{\text{safe}}(z) = r_{\text{part}}(z) + r + \text{margin}$ on a circular orbit (for the sphere $r_{\text{part}}(z)$ varies with z; for the cylinder it is z-invariant).
- **Box:** a *square* orbit $(\cos\theta, \sin\theta)/\max(|\cos\theta|, |\sin\theta|)$ at $r_{\text{safe}} = r_{\text{part}} + r + \text{margin}$, sweeping just outside the box faces; a circular orbit would under-cover the corners.
- **Pyramid:** a four-phase hybrid — (i) an above-apex boustrophedon clearing the top slab, (ii) a square-annulus orbit with base descending apex-to-base as the safe radius shrinks with the pyramid's cross-section, (iii) a circular safe-radius descent clearing the lower annulus, and (iv) a fixed-low-base boustrophedon clearing the below-pyramid slab. Phase (iv) relies on the fact that the deployable carve uses only the tool SDF (Eq. 7); setting the base so the tool top sits just below the pyramid base carves the entire below-slab without gouging, since the holder does not participate in the boolean union.

The per-shape parameters (R, ω, margin, phase budgets) are found by a fast geometry search that scores candidate trajectories directly under the hard carve (Sec. 1.9) in seconds per configuration, without running the optimizer. For the sphere, Dice scales with T (each z-level has a distinct $r_{\text{part}}(z)$ needing its own angular coverage); for the z-invariant cylinder and box cross-sections, a fixed revolution count already saturates a structural ceiling.

1.8 Best-Checkpoint Selection

Because soft optimization degrades the deployable metric (Sec. 1.6), the deployed trajectory is not the final iterate. Every K iterations (default K=25) the trainer evaluates the current trajectory under the hard carve (Eq. 7) from the canonical start and records the best-Dice trajectory and its measured Dice. The reported and deployed result is this best checkpoint. For the geometry-informed initializations, the best checkpoint is at iteration 0 — the init itself — since soft optimization only moves away from it. We do not re-evaluate restored parameters at the end (the hard carve has small nondeterminism under GPU atomic-add reductions); instead we store the exact best-iteration positions and the Dice measured at that iteration, so the reported number corresponds to the saved trajectory.

1.9 Evaluation Protocol

All reported metrics use the exact boolean carve (Eq. 7) with speed clipping enabled, matching the deployable machining constraint. We report Dice, average surface distance (ASD), and 95th-percentile Hausdorff distance (HD95) between the remaining material ($\Phi^{\text{hard}} < 0$) and the target part. The method is deterministic: from a fixed initialization the hard carve of the saved trajectory is identical across seeds (the optimization seed affects only the discarded soft refinement), so the reported gains are exact rather than within-seed variance.

1.10 Summary

GradMill optimizes a CNC trajectory by backpropagation through a differentiable CSG simulator. The simulator's gradients are correctly computed and applied each iteration, but they minimize a soft surrogate whose over-erosion bias makes it misaligned with the deployable boolean outcome; following them degrades the deployable metric. The method therefore rests on three pillars: (1) a differentiable carving simulator that defines a joint objective over a structured trajectory space, (2) a shape-aware z-level initialization that encodes machining domain knowledge and sets the deployable optimum, and (3) a best-checkpoint protocol that preserves that optimum against surrogate-driven degradation. The differentiable simulator enables the search (a smooth objective, fast geometry scoring, and gradient-based exploration) even where its gradients are not the final instrument of refinement.